

**Tecnologico Superior
De Rio Verde**

Practica10

Profesor: Jose Jesus Collazo.

Alumno: Issac Martin Recendiz Hernandez

Grupo: 2A

Fecha: 27 de Abril de 2026

INDICE

- Introducción
- Planteamiento del problema
- Diseño de la solución
- Desarrollo (código explicado)
- Evidencia de ejecución
- Análisis del programa
- Conclusiones

INTRODUCCIÓN

La programación orientada a objetos (POO) es un paradigma ampliamente utilizado en el desarrollo de software, ya que permite organizar los programas de una manera más estructurada, clara y reutilizable. A diferencia de otros enfoques, la POO se basa en el uso de clases y objetos para representar entidades del mundo real, lo cual facilita la comprensión de los sistemas y su mantenimiento a lo largo del tiempo.

Uno de los conceptos más importantes dentro de este paradigma es la herencia. La herencia permite que una clase pueda adquirir atributos y métodos de otra clase ya existente, conocida como clase base. Esto no solo ayuda a evitar la duplicación de código, sino que también permite establecer relaciones jerárquicas entre diferentes clases, haciendo que el diseño del programa sea más lógico y organizado. Gracias a este mecanismo, es posible extender funcionalidades sin necesidad de modificar directamente el código original.

En el desarrollo de software, la herencia es especialmente útil cuando se trabaja con entidades que comparten características en común, pero que también presentan diferencias específicas. En lugar de crear clases completamente independientes, se puede definir una clase general que contenga los elementos comunes, y posteriormente crear clases más específicas que hereden de esta y agreguen o modifiquen comportamientos según sea necesario. Esto no solo mejora la reutilización del código, sino que también facilita futuras modificaciones o ampliaciones del sistema.

En esta práctica, se aplica el concepto de herencia para modelar un sistema de empleados dentro de una empresa. Todos los empleados comparten información básica, como su nombre, edad y salario, pero existen diferentes tipos de empleados con características particulares. Por ejemplo, algunos empleados trabajan tiempo completo y reciben un bono adicional, mientras que otros trabajan por horas y su salario depende del tiempo trabajado. Asimismo, existen empleados que cuentan con prestaciones adicionales, como seguro o días de vacaciones.

El objetivo de este trabajo es comprender cómo se puede utilizar la herencia para representar estas diferencias de manera eficiente, evitando repetir código y manteniendo una estructura clara. Además, se busca reforzar otros conceptos importantes como el uso de constructores, la sobrescritura de métodos y la organización adecuada de clases en un programa.

Finalmente, este tipo de ejercicios permite desarrollar una mejor lógica de programación y entender cómo aplicar los conceptos teóricos en situaciones prácticas. La herencia no solo es un tema importante en el ámbito académico, sino que también es una herramienta fundamental en el desarrollo profesional, ya que se utiliza en la creación de sistemas complejos, aplicaciones empresariales y diversos tipos de software en la vida real.

PLANTEAMIENTO DEL PROBLEMA

El problema consiste en desarrollar un programa en C++ que permita representar diferentes tipos de empleados dentro de una empresa.

Todos los empleados tienen información básica como nombre, edad y salario. Sin embargo, no todos se comportan igual, ya que algunos trabajan tiempo completo y reciben un bono, mientras que otros trabajan por horas y su salario depende del tiempo trabajado.

Además, existen empleados que cuentan con prestaciones adicionales como seguro y vacaciones, lo que hace necesario agregar más características al modelo.

El objetivo principal es aplicar la herencia para organizar estas diferencias sin repetir código, creando una estructura clara que permita representar cada tipo de empleado de forma adecuada.

DISEÑO DE LA SOLUCIÓN

Antes de comenzar a programar, se analizó el problema para identificar qué elementos eran comunes y cuáles eran diferentes entre los empleados. Con base en esto, se decidió utilizar una clase base y varias clases derivadas.

● **Clase base: Empleado**

Se creó una clase base llamada **Empleado**, la cual contiene los atributos comunes:

- Nombre
- Edad
- Salario

También incluye métodos como:

- Mostrar información
- Calcular salario

● **Clases derivadas**

Se implementaron dos clases derivadas:

EmpleadoTiempoCompleto

- Hereda de Empleado
- Agrega el atributo bono
- Sobrescribe el método de salario para incluir el bono

EmpleadoPorHoras

- Hereda de Empleado
- Incluye horas trabajadas y pago por hora
- Calcula el salario en función de estos valores

● **Herencia múltiple**

Se creó una clase adicional llamada **Prestaciones**, que contiene:

- Seguro
- Días de vacaciones

Posteriormente, se creó la clase:

EmpleadoCompletoConPrestaciones

- Hereda de EmpleadoTiempoCompleto y Prestaciones
- Combina salario con beneficios adicionales

Relación entre clases

- Empleado → clase base
- EmpleadoTiempoCompleto → hereda de Empleado
- EmpleadoPorHoras → hereda de Empleado
- Prestaciones → clase independiente
- EmpleadoCompletoConPrestaciones → herencia múltiple

Este diseño permite reutilizar código y extender funcionalidades fácilmente.

DESARROLLO

El código fue desarrollado paso a paso, comenzando por la clase base y posteriormente agregando las clases derivadas. Durante el proceso se realizaron pruebas para verificar que el cálculo del salario funcionara correctamente en cada tipo de empleado.

◆ **Constructor**

Cada clase tiene un constructor que inicializa sus atributos. En las clases derivadas, se utiliza la herencia para llamar al constructor de la clase base.

◆ Herencia

Se utiliza la sintaxis:

```
class EmpleadoTiempoCompleto : public Empleado
```

Esto indica que la clase hereda los atributos y métodos de la clase base.

◆ Override

Se utiliza la palabra clave override para redefinir métodos heredados, como el cálculo del salario, permitiendo que cada tipo de empleado tenga un comportamiento distinto.

◆ Métodos

Cada clase tiene métodos para mostrar información y calcular el salario, adaptados según el tipo de empleado.

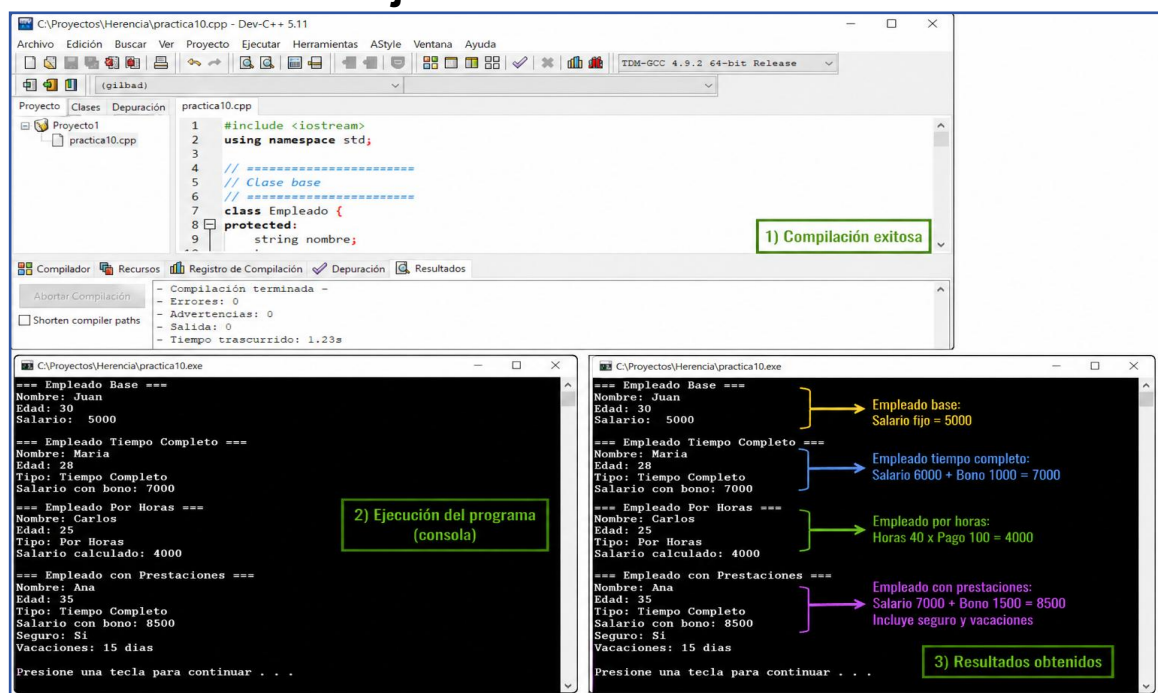
◆ Función main()

En la función principal se crean objetos de cada tipo de empleado:

- Empleado base
- Empleado de tiempo completo
- Empleado por horas
- Empleado con prestaciones

Posteriormente, se muestran sus datos y se calcula su salario

● Evidencia de Ejecucion



ANÁLISIS DEL PROGRAMA

El uso de herencia facilitó mucho la organización del programa, ya que permitió definir una sola vez los atributos comunes y reutilizarlos en las demás clases. Esto hizo que el código fuera más limpio y entendible.

Si no se hubiera utilizado herencia, habría sido necesario repetir varias partes del código en cada tipo de empleado, lo cual aumentaría la probabilidad de errores.

Una de las dificultades fue entender cómo estructurar correctamente las clases y cuándo era necesario sobrescribir métodos. Sin embargo, una vez comprendido esto, el desarrollo del programa resultó más sencillo.

También se observó que la herencia múltiple puede ser útil, pero debe utilizarse con cuidado para no complicar demasiado el diseño.

CONCLUSIONES

En esta práctica se logró comprender de manera más clara el uso de la herencia dentro de la programación orientada a objetos. A través del desarrollo del programa, fue posible identificar cómo una clase base puede servir como punto de partida para crear otras clases más específicas, permitiendo reutilizar código y mantener una mejor organización. Esto facilitó la implementación de distintos tipos de empleados sin necesidad de repetir estructuras similares.

Además, se pudo observar que la herencia no solo ayuda a reducir la cantidad de código, sino que también mejora la legibilidad del programa y su mantenimiento. Al tener una estructura bien definida, resulta más sencillo realizar modificaciones o agregar nuevas funcionalidades en el futuro, como podría ser la incorporación de nuevos tipos de empleados con características distintas.

Durante el desarrollo, uno de los aspectos más importantes fue entender cuándo es necesario sobrescribir métodos y cómo adaptar el comportamiento de cada clase derivada. Esto permitió que cada tipo de empleado tuviera su propia forma de calcular el salario, lo cual es fundamental para representar correctamente el problema planteado.

Finalmente, esta práctica ayudó a reforzar la lógica de programación y a comprender la importancia de diseñar adecuadamente las clases antes de comenzar a programar. Este conocimiento resulta útil no solo en el ámbito académico, sino también en aplicaciones reales donde se requiere desarrollar sistemas más estructurados, escalables y fáciles de mantener.